

CSE357: COMBINATORIAL STUDIES

Dr. Harwinder Singh Sohal

Unit II: Database Management Systems

- Introduction to Databases and RDBMS
- Basics of Databases and RDBMS
- Data Definitions: Tables, Fields, Records, SQL and Data Manipulation, Database Keys and Data Integrity
- Database Normalization and Transactions
- Fundamentals of Normalization
- Transaction Management in DBMS

Introduction to Database Management Systems(DBMS)

- Definition of Databases and Database Management Systems
- Key Components of DBMS
 - Data Definition Language (DDL)
 - Data Manipulation Language (DML)
 - Data Query Language (DQL)
 - Data Control Language (DCL)
 - Transaction Management
 - Concurrency Control
 - Backup and Recovery



A **database** is an organized collection of data that can be easily accessed, managed, and updated. It allows users to store information in a structured way, so it can be efficiently retrieved and manipulated.

Key Points:

- Stores **large amounts of data** systematically.
- Supports **data management operations**: insert, update, delete, and retrieve.
- Eliminates data **redundancy** and improves **data integrity**.

Types of Databases

1.Hierarchical Database: Data is organized in a tree-like structure.

Example: IBM IMS.

2.Network Database: Uses graph structures where nodes are connected via relationships.

Example: Integrated Data Store (IDS).

3.Relational Database: Data is stored in **tables** (rows and columns) with relationships.

Example: MySQL, Oracle, SQL Server.

4.Object-Oriented Database: Stores data as objects, similar to object-oriented programming.

Example: db4o, ObjectDB.

Example:

- A college database storing student records (Name, Roll No., Course, Marks).
- A bank database storing account information (Account No., Name, Balance, Transactions).

Introduction to Database Management Systems(DBMS) :

A Database Management System (DBMS) is software used to manage data from a database. It acts as an interface between the database and end users or applications, ensuring data is consistently organized, easily accessible, and secure.

- **Benefits of Using DBMS**

- Data Centralization
- Data Security
- Data Integrity
- Data Scalability
- Data Recovery



Applications of DBMS

Database Design

- the process of defining the structure and organization of a database to efficiently store, manage, and retrieve data.
- Key Components of Database Design
 - Entity-relationship (ER) Modeling
 - Database Schema
 - Normalization
 - Indexing
 - Data Integrity Constraints

Example of a Relation

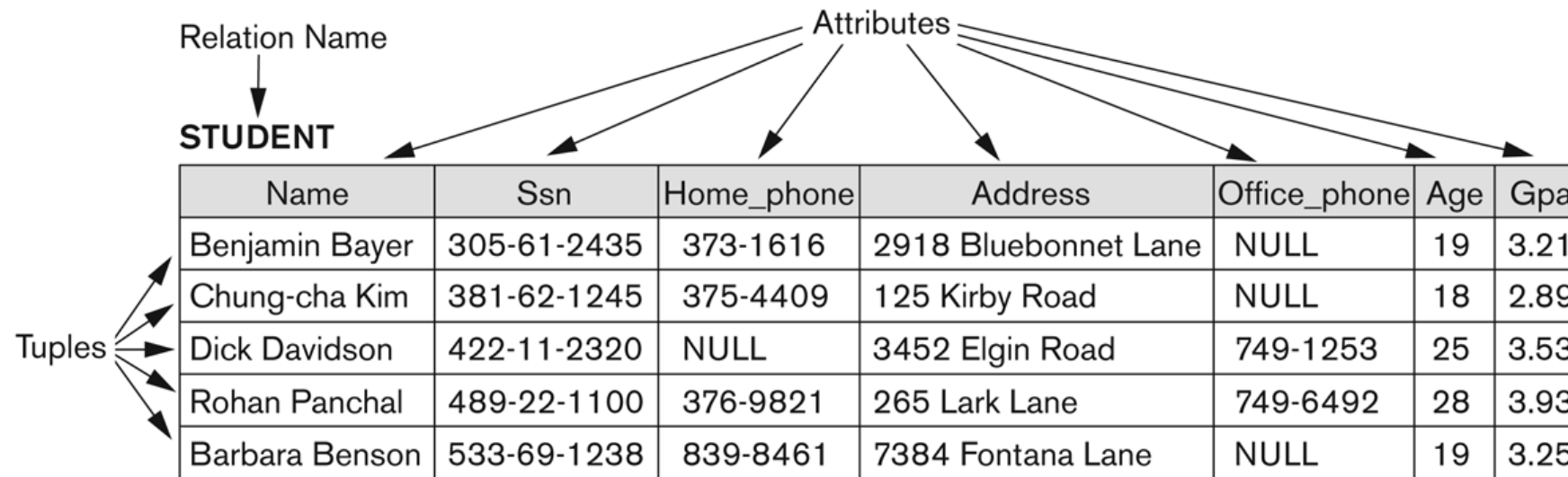
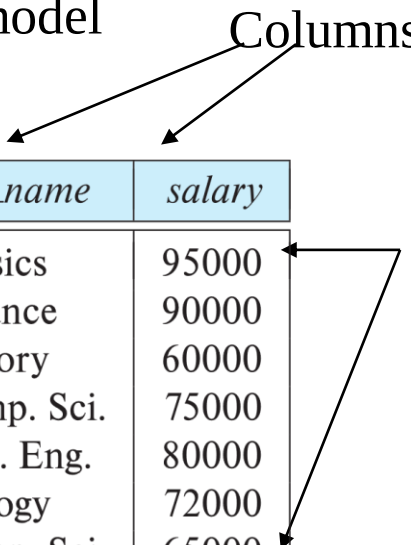


Figure 5.1

The attributes and tuples of a relation STUDENT.

Relational Model

- All the data is stored in various tables.
- Example of tabular data in the relational model



<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

A Sample Relational Database

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

- **Types of Database Management Systems**
 - Relational Database Management Systems(RDBMS)
 - NoSQL DBMS
 - NewSQL DBMS

Basics of Relational Database Management Systems (RDBMS)

- Introduction to RDBMS
- Key Concepts of RDBMS
 - Tables
 - Primary Key
 - Foreign Key
 - Relationships
 - Normalization
 - Structured Query Language(SQL)
 - ACID Properties



Basics of Relational Database Management Systems (RDBMS)

- It is a type of database management system that stores data in the form of **tables (relations)** consisting of **rows (records/tuples)** and **columns (fields/attributes)**.
- RDBMS allows users to **store, manage, retrieve, and manipulate data efficiently** using **SQL (Structured Query Language)**.

Key Features of RDBMS

- 1.Data stored in tables** (rows & columns)
- 2.Primary and foreign keys** to establish relationships
- 3.Supports SQL** for data manipulation
- 4.Ensures data integrity** with constraints
- 5.Allows multiple users** to access the database concurrently
- 6.Provides security, backup, and recovery**

Keys in Relational Model

Keys are fundamental elements of the relational database model that ensure uniqueness, data integrity, and efficient data access.

- They uniquely identify each row in a table.
- They prevent data duplication and maintain consistency.
- They create relationships between different tables.

Student

StudentID	RollNo	Email	Name	DeptID
101	01	a@gmail.com	Amit	D1
102	02	b@gmail.com	Neha	D2
103	03	c@gmail.com	Rohit	D1

Primary Key?

A **Primary Key (PK)** is an attribute (or a combination of attributes) that **uniquely identifies each record (row)** in a table.

Properties of a Primary Key:

- Must be **unique**
- Must **not contain NULL values**
- Should be **stable (not change frequently)**
- Should be **minimal** (no unnecessary attributes)

Foreign Key?

A **Foreign Key (FK)** is an attribute (or a combination of attributes) in one table that **refers to the Primary Key of another table.**

Purpose of Foreign Key

- Maintains **referential integrity**
- Establishes **relationship between tables**
- Prevents **invalid data entry**

STUDENT Table

StudentID	RollNo	Email	Name	DeptID
101	01	a@gmail.com ↗	Amit	D1
102	02	b@gmail.com ↗	Neha	D2
103	03	c@gmail.com ↗	Rohit	D1

DEPARTMENT Table

DeptID	DeptName
D1	CSE
D2	IT

Composite Foreign Key → (RollNo, DeptID)

Case: When used?

If another table like **RESULT** exists:

RESULT Table

RollNo	DeptID	Marks
01	D1	85
02	D2	78

Super Key

- A Super Key is any single attribute or combination of attributes that can uniquely identify a record (row) in a table.
- A super key may contain extra attributes
- Every primary key is a super key
- But not every super key is a primary key

Single Attribute Super Keys

Attribute	Reason
StudentID	Always unique
Email	Unique for each student

Composite Super Keys (Attribute Combinations)

Combination	Reason
StudentID + Name	Still unique
StudentID + DeptID	Still unique
StudentID + Email	Still unique
RollNo + DeptID	Unique together
Email + DeptID	Still unique

Candidate Key?

A **Candidate Key** is a **minimal set of attributes that can uniquely identify each record (row) in a table.**

Property	Description
Uniqueness	Must uniquely identify each record
Minimal	No extra attribute is allowed
NULL values	✗ Not allowed
Count per table	Can be more than one
Subset of super key	Yes
Superset of primary key	No (primary key is selected from candidate keys)

Step 1: Identify Super Keys

- StudentID
 - Email
 - RollNo + DeptID
 - StudentID + Name
 - StudentID + DeptID
-

Step 2: Remove Non-Minimal Keys

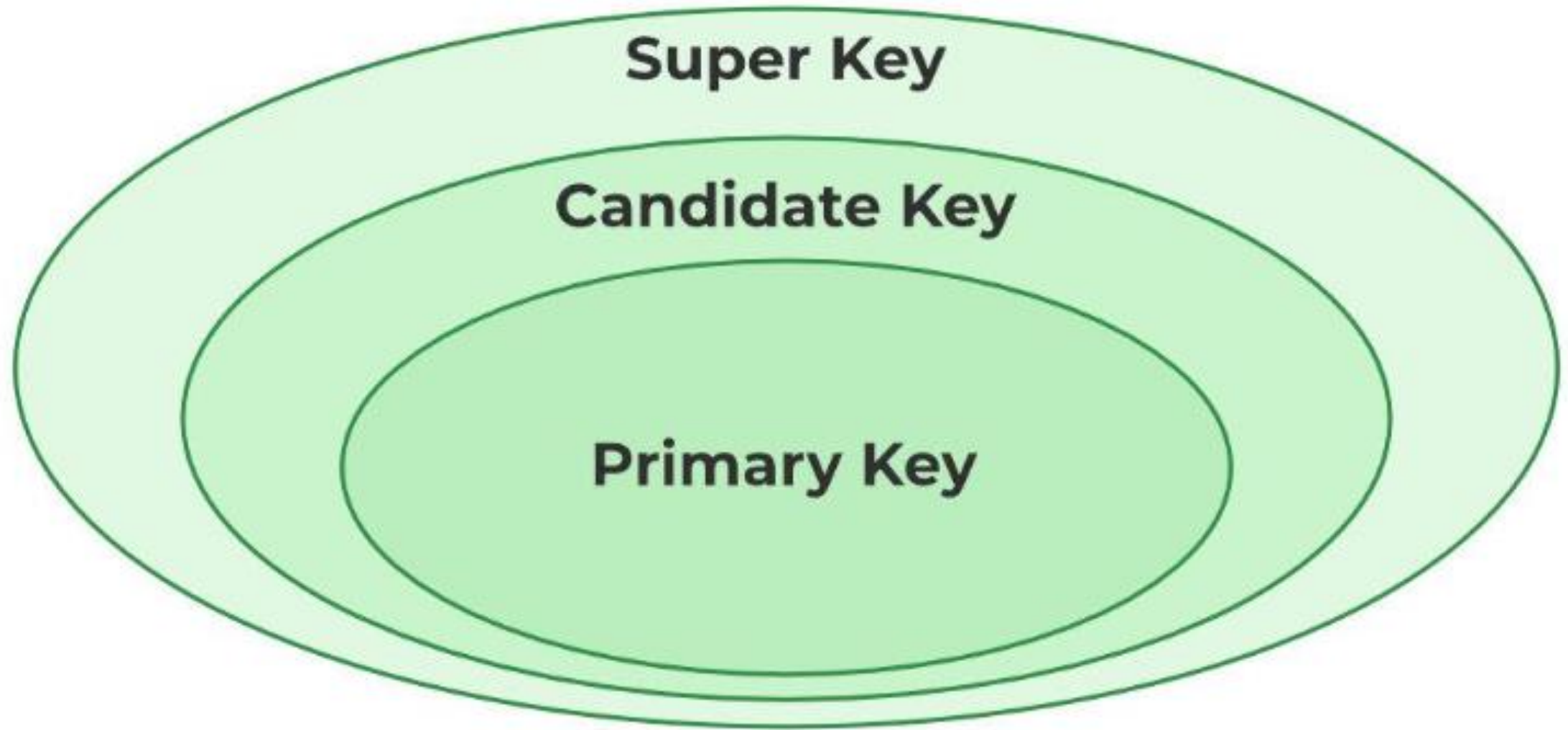
Remove combinations that contain extra attributes:

- StudentID + Name ✗ (StudentID alone is enough)
- StudentID + DeptID ✗ (StudentID alone is enough)

Candidate Key

Candidate Key	Reason
StudentID	Unique, minimal, stable
Email	Unique, minimal
RollNo + DeptID	Unique combination, minimal

Best Choice: StudentID



Relation between Primary Key, Candidate Key, and Super Key

Alternate Key

An **Alternate Key** is a **candidate key** that is **NOT** selected as the **primary key**.

Candidate Keys:

- StudentID
- Email
- (RollNo + DeptID)

Selected Primary Key:

- StudentID

Therefore, Alternate Keys:

- Email
- (RollNo + DeptID)

Feature	Alternate Key
Uniqueness	Yes
NULL allowed	× No
Count	One or more
Example	Email

Composite Key

A composite key is a combination of two or more attributes that together uniquely identify a tuple.

Example

RollNo	DeptID
01	D1
01	D2

Here:

- RollNo alone ✗ not unique
- DeptID alone ✗ not unique
- RollNo + DeptID → Unique

So:

👉 (RollNo, DeptID) → Composite Key

Feature	Composite Key
No. of attributes	≥ 2
Uniqueness	Yes
NULL allowed	× No
Example	(RollNo, DeptID)

Key Type	Unique	Minimal	NULL Allowed	Example
Super Key	Yes	✗ No	⚠ Yes	(StudentID + Name)
Candidate Key	Yes	✓ Yes	✗ No	Email
Primary Key	Yes	✓ Yes	✗ No	StudentID
Alternate Key	Yes	✓ Yes	✗ No	Email
Composite Key	Yes	✓ Yes	✗ No	(RollNo + DeptID)

Formula of Super Keys using Candidate Key

If:

- n = Total number of attributes in relation
- k = Number of attributes in candidate key

Then:

$$\text{Number of Super Keys} = 2^{(n-k)}$$

All supersets of candidate key = Super Keys

Relation **R(RollNo, RegNo, Name, Dept)**

Candidate Key = **(RollNo, RegNo)**

So:

$$n = 4 \quad k = 2$$

$$\text{Super Keys} = 2^{\{(4 - 2)\}} = 4$$

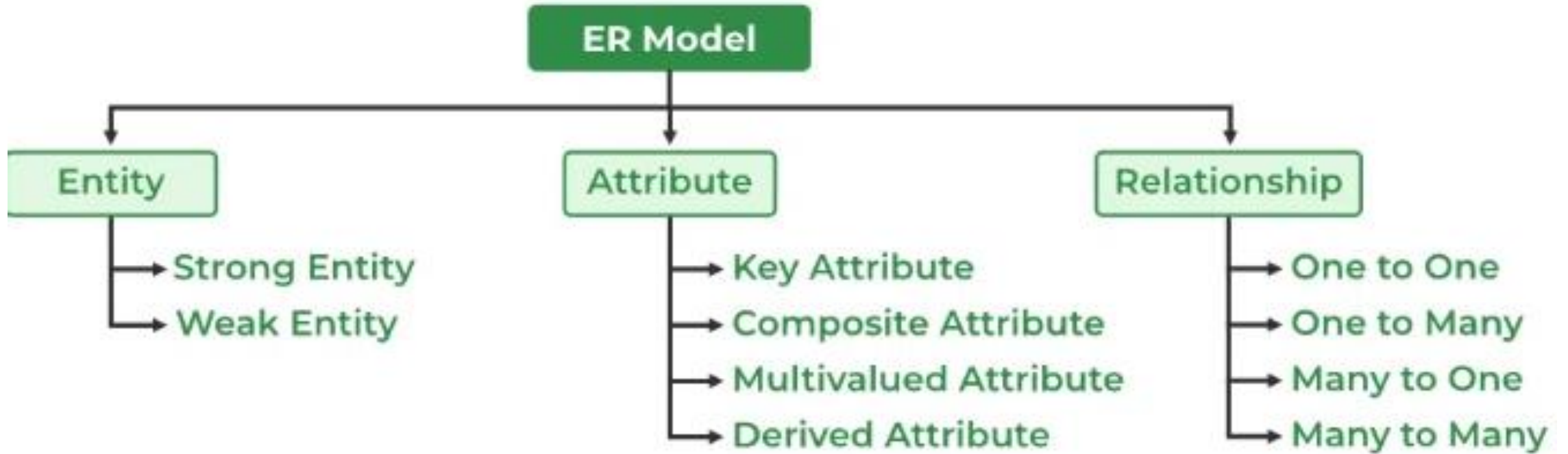
Super Keys:

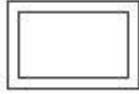
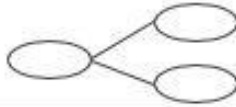
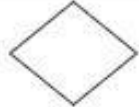
- (RollNo, RegNo)
- (RollNo, RegNo, Name)
- (RollNo, RegNo, Dept)
- (RollNo, RegNo, Name, Dept)

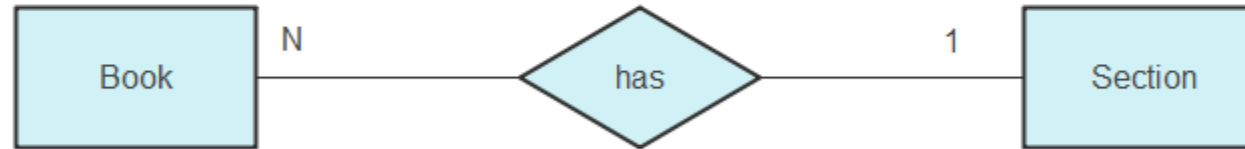
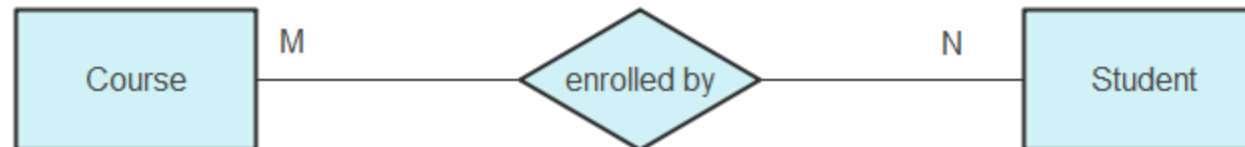
Introduction of ER Model

The Entity-Relationship Model (ER Model) is a conceptual model for designing a database. This model represents the logical structure of a database, including entities, their attributes, and relationships between them.

- **Entity:** An object that is stored as data. E.g: *Student, Course, or Company*.
- **Attribute:** Properties that describe an entity. E.g: *StudentID, CourseName, or EmployeeEmail*.
- **Relationship:** A connection between entities. E.g: *Student enrolls in a Course*.



Entity Set 	Strong Entity Set	
	Weak Entity Set	
Attributes 	Simple Attribute	
	Composite Attribute	
	Single-valued Attribute	
	Multivalued Attribute	
	Derived Attribute	
	Null Attribute	
Relationship	Strong Relationship	
	Weak Relationship	

one-to-one (1:1)**one-to-many (1:N)****many-to-one (N:1)****many-to-many (M:N)**

STUDENT

StudentID	RollNo	Aadhaar	Email	Name	DeptID
101	01	123456	a@gmail.com	Amit	D1
102	02	789456	b@gmail.com	Neha	D2
103	03	654987	c@gmail.com	Rohit	D1

1. Functional Dependency (FD)

A Functional Dependency means:

If we know **A**, then we can uniquely determine **B**.

Example:

nginx

StudentID → Name

Meaning:

- If StudentID = 101 → Name = Amit
- If StudentID = 102 → Name = Neha

2. Full Functional Dependency

A Full Functional Dependency means:

An attribute depends on the entire primary key, not on a part of it.

Here, Primary Key = StudentID (single attribute)

So:

nginx

StudentID → Name

StudentID → Email

StudentID → DeptID

All attributes fully depend on StudentID.

✓ So, ALL dependencies here are FULL functional dependencies.

3. Partial Dependency

Partial Dependency occurs when:

A non-prime attribute depends on part of a composite primary key.

But here:

- Primary key = **StudentID** only
- Not composite (not multi-column)

So:

✗ NO Partial Dependency exists in this table

4. Transitive Dependency

A Transitive Dependency occurs when:

less

$A \rightarrow B$

$B \rightarrow C$

Then: $A \rightarrow C$ (indirect dependency)

$StudentID \rightarrow DeptID$

$DeptID \rightarrow DepartmentName$

$StudentID \rightarrow DepartmentName$

This becomes **TRANSITIVE DEPENDENCY**.

1. Functional Dependency (FD) Recap

A Functional Dependency $X \rightarrow Y$ means:

"If we know X , we can uniquely determine Y ."

Where:

- X = Determinant (LHS)
- Y = Dependent attribute(s) (RHS)

2. Trivial Functional Dependency

Definition:

A functional dependency $X \rightarrow Y$ is **trivial** if $Y \subseteq X$.

Key Idea:

- RHS is **already part of LHS** $\rightarrow$ no new information.

Example:

FD	Why Trivial?
$AB \rightarrow A$	A is part of AB
$CDE \rightarrow D$	D is part of CDE

3. Non-Trivial Functional Dependency

Definition:

A functional dependency $X \rightarrow Y$ is **non-trivial** if $Y \not\subseteq X$ and $Y \cap X = \emptyset$ (no common attribute).

Key Idea:

- RHS has **completely new attributes** not in LHS.

Example:

FD	Why Non-Trivial?
$AB \rightarrow CD$	C and D are not in AB
$E \rightarrow F$	F is not in E

4. Semi Non-Trivial Functional Dependency

Definition:

A functional dependency $X \rightarrow Y$ is semi non-trivial if $Y \cap X \neq \emptyset$ but $Y \not\subseteq X$.

Key Idea:

- RHS shares some attributes with LHS, but also has new attributes.

Example:

FD

Why Semi Non-Trivial?

$AB \rightarrow BC$

B is common, C is new

$XY \rightarrow YZ$

Y is common, Z is new

Quick Summary Table

Type	Condition	Example
Trivial	$RHS \subseteq LHS$	$AB \rightarrow A, CDE \rightarrow D$
Non-Trivial	$RHS \cap LHS = \emptyset$	$AB \rightarrow CD, E \rightarrow F$
Semi Non-Trivial	$RHS \cap LHS \neq \emptyset,$ $RHS \not\subseteq LHS$	$AB \rightarrow BC, XY \rightarrow YZ$

 EXAMPLE 1

Given:

$$R(A, B, C, D, E, F, G)$$

Functional Dependencies:

- $A \rightarrow B$
- $B \rightarrow C$
- $D \rightarrow E$
- $AC \rightarrow F$
- $F \rightarrow G$

Question:

Which of the following is the **candidate key**?

- A. A
- B. AD
- C. AC
- D. CD

Short Trick

1. Find **attributes not on RHS** → must be in key

2. Here: **A, D**

☞ So **AD** is the candidate key

 EXAMPLE 2

Given:

$$R(P, Q, R, S, T, U, V)$$

Functional Dependencies:

- $P \rightarrow Q$
- $Q \rightarrow R$
- $R \rightarrow S$
- $S \rightarrow T$
- $T \rightarrow U$

Question:

Which of the following is the **candidate key**?

- A. P
- B. Q
- C. PR
- D. PV

Normalization

- the process to reduce data redundancy, improve data integrity, and optimize database performance.
- Types of Normalization
 - First Normal Form (1NF)
 - Second Normal Form (2NF)
 - Third Normal Form (3NF)
 - Boyce-Codd Normal Form (BCNF)
 - Fourth Normal Form (4NF)
 - Fifth Normal Form (5NF)



Normalization?

Normalization is the process of organizing data in a database to:

- Reduce **data redundancy**
- Avoid **update, insert, and delete anomalies**
- Improve **data integrity and consistency**
- It is done by **decomposing (splitting) tables** based on rules called **Normal Forms**.

Unnormalized Form (UNF)

- Raw table structure with all data combined.
- May contain **repeating groups or arrays** in a single column.
- No primary key is defined.
- High **redundancy** (duplicate rows).
- Prone to **insertion, update, and deletion anomalies**.
- Example: STUDENT table with Course, Student, System, Hours, Rate all in one table.

First Normal Form (1NF)

- All attributes must be **atomic** (indivisible).
- No **repeating groups** or arrays in a single column.
- Define a **primary key** (can be composite).
- Eliminates **repetition within a row**.
- Reduces some redundancy, but partial and transitive dependencies may remain.

1 First Normal Form (1NF)

✓ Rule:

- Atomic values (no multi-valued attributes)
- No repeating groups

◆ Dependency Concept:

- Remove multi-valued dependencies

✗ Before 1NF:

Roll	Name	Phone
1	A	9876, 8765

✓ After 1NF:

Roll	Name	Phone
1	A	9876
1	A	8765

Anomalies in 1NF:

Anomaly	Exists?
Insertion	✓
Deletion	✓
Update	✓

👉 1NF removes repeating values but NOT anomalies fully.

2 Second Normal Form (2NF)

✓ Rule:

- Must be in 1NF
- No partial dependency

◆ Partial Dependency:

When non-prime attribute depends on part of composite key

Example:

Roll	Subject	StudentName	Marks
1	Math	A	90
1	Sci	A	85

Key = (Roll, Subject)

✓ Convert to 2NF:

Student Table

Roll	StudentName
------	-------------

Marks Table

Roll	Subject	Marks
------	---------	-------

● Anomalies in 2NF:

Anomaly	Exists?
---------	---------

Insertion	✓
-----------	---

Deletion	✓
----------	---

Update	✓
--------	---

👉 2NF removes partial dependency but transitive dependency remains.

3 Third Normal Form (3NF)

✓ Rule:

- Must be in 2NF
- No transitive dependency

◆ Transitive Dependency:

$A \rightarrow B$ and $B \rightarrow C$

So, $A \rightarrow C$ (indirect)

Example:

Roll	Name	DeptID	DeptName
------	------	--------	----------

FD:

nginx

Roll → Name, DeptID

DeptID → DeptName

Transitive:

nginx

Roll → DeptName ❌

✔ Convert to 3NF:

Student Table

| Roll | Name | DeptID |

Department Table

| DeptID | DeptName |

● Anomalies in 3NF:

Anomaly	Exists?
Insertion	✗
Deletion	✗
Update	✗

👉 3NF removes all major anomalies.

Database Normalization

1NF



- Data is stored in tabular format
- Each column holds atomic values (no lists)
- No repeating groups or arrays

2NF



- Meets 1NF
- No partial dependencies on composite primary keys
- Data split into smaller tables

3NF



- Meets 2NF
- No transitive dependencies
- Each column depends only on the primary key

4 Boyce–Codd Normal Form (BCNF)

✓ Rule:

For every functional dependency $X \rightarrow Y$,
X must be a super key

Example:

Student	Course	Teacher
---------	--------	---------

FD:

SCSS

$(\text{Student}, \text{Course}) \rightarrow \text{Teacher}$

$\text{Teacher} \rightarrow \text{Course}$ ✗ (Teacher not super key)

✗ Violates BCNF

But table **may still be in 3NF**

✓ BCNF Decomposition:

Teacher Table

| Teacher | Course |

Enrollment Table

| Student | Teacher |

● Anomalies in BCNF:

Anomaly	Exists?
Insertion	✗
Deletion	✗
Update	✗

👉 BCNF gives stronger dependency control than 3NF.

5 Fourth Normal Form (4NF)

✓ Rule:

- Must be in BCNF
- No multi-valued dependency

Example:

Student	Phone	Skill
---------	-------	-------

MVD:

nginx

Student $\twoheadrightarrow$ Phone

Student $\twoheadrightarrow$ Skill

✔ Convert to 4NF:

Phone Table

| Student | Phone |

Skill Table

| Student | Skill |

● Anomalies in 4NF:

Anomaly	Exists?
Insertion	✗
Deletion	✗
Update	✗

6 Fifth Normal Form (5NF)

✓ Rule:

- Remove join dependency
 - Table cannot be decomposed further without loss
-

Used when:

- Complex many-to-many-to-many relationships exist.

(Usually beyond UG syllabus, asked rarely)

Normal Form	Dependency Removed	Anomalies Removed
1NF	Multivalued	✗
2NF	Partial dependency	Partial
3NF	Transitive dependency	✓
BCNF	Non-key dependency	✓
4NF	Multivalued dependency	✓
5NF	Join dependency	✓

1NF $\rightarrow$ Atomic values

2NF $\rightarrow$ No Partial dependency

3NF $\rightarrow$ No Transitive dependency

BCNF $\rightarrow$ Determinant must be Super Key

4NF $\rightarrow$ No Multivalued dependency

5NF $\rightarrow$ No Join dependency

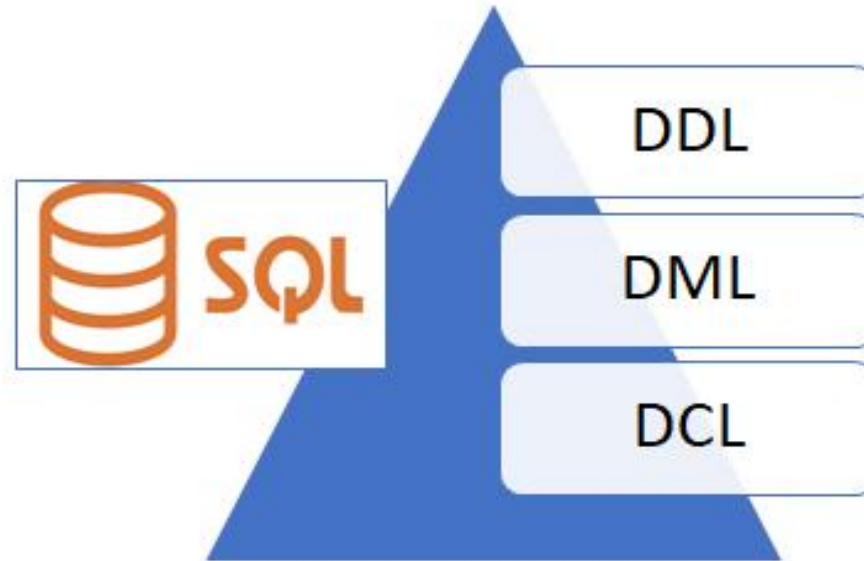
Structure Query language(SQL)

- Introduction to SQL
- DDL (Data Definition Language)
 - Create Table
 - Alter Table
 - Drop Table
 - Create Index
 - Drop Index
 - Create Database
 - Alter Database
 - Drop Database



Structure Query language(SQL)

- DML (Data Manipulation Language):
 - Select
 - Insert Into
 - Update
 - Delete From
 - Merge Into
 - Truncate Table
 - Copy
 - Fetch



Structure Query language(SQL)

- DCL (Data Control Language)
 - Grant
 - Revoke
- SQL Syntax:
 - SELECT Statement:
 - SELECT column1, column2, ...
 - FROM table_name
 - WHERE condition;
 - INSERT INTO Statement:
 - INSERT INTO table_name (column1, column2, ...)
 - VALUES (value1, value2, ...);
 - UPDATE Statement:
 - UPDATE table_name
 - SET column1 = value1, column2 = value2, ...
 - WHERE condition;
 - DELETE FROM Statement:
 - DELETE FROM table_name
 - WHERE condition;

EMPNO	ENAME	JOB	HIREDATE	MGR	SAL	COMM	DEPTNO
7369	SMITH	CLERK	17-DEC-80	7902	800		20
7499	ALLEN	SALESMAN	20-FEB-81	7698	1600	300	30
7521	WARD	SALESMAN	22-FEB-81	7698	1250	500	30
7566	JONES	MANAGER	02-APR-81	7839	2975		20
7654	MARTIN	SALESMAN	28-SEP-81	7698	1250	1400	30
7698	BLAKE	MANAGER	01-MAY-81	7839	2850		30
7782	CLARK	MANAGER	09-JUN-81	7839	2450		10
7788	SCOTT	ANALYST	19-APR-87	7566	3000		20
7839	KING	PRESIDENT	17-NOV-81		5000		10
7844	TURNER	SALESMAN	08-SEP-81	7698	1500	0	30
7876	ADAMS	CLERK	23-MAY-87	7788	1100		20
7900	JAMES	CLERK	03-DEC-81	7698	950		30
7902	FORD	ANALYST	03-DEC-81	7566	3000		20
7934	MILLER	CLERK	23-JAN-82	7782	1300		10

Questions

1. Write an SQL query to **display the names of all employees working in department 30.**
2. Write an SQL query to **find the names and salaries of employees whose salary is greater than 2500.**
3. Write an SQL query to **display the employee name, job, and hire date of all managers.**
4. Write an SQL query to **find the total number of employees in the EMP table.**
5. Write an SQL query to **display employee names who are working as SALESMAN.**

Questions

1. Write an SQL query to **display the names of all employees working in department 30.**
2. Write an SQL query to **find the names and salaries of employees whose salary is greater than 2500.**
3. Write an SQL query to **display the employee name, job, and hire date of all managers.**
4. Write an SQL query to **find the total number of employees in the EMP table.**
5. Write an SQL query to **display employee names who are working as SALESMAN.**

•Write an SQL query to **display the number of employees in each department**

using `GROUP BY` .

•Write an SQL query to **find the average salary of employees in each department.**

•Write an SQL query to **display departments having more than 3 employees**

using `GROUP BY` and `HAVING` .

•Write an SQL query to **find the maximum salary in each department.**

•Write an SQL query to **display employee names along with their manager**

names using a **self JOIN**.

- Write an SQL query to **display department number and total salary paid in each department.**
- Write an SQL query to **display departments where the average salary is greater than 2000** using HAVING .
- Write an SQL query to **find the job title and the number of employees working in each job.**
- Write an SQL query to **display job titles having more than 2 employees** using GROUP BY and HAVING .
- Write an SQL query to **display employee names, salary, and department number for employees whose salary is greater than the average salary of all employees.**

6 Write an SQL query to **find the highest salary in the EMP table.**

7 Write an SQL query to **display employee name and salary in descending order of salary.**

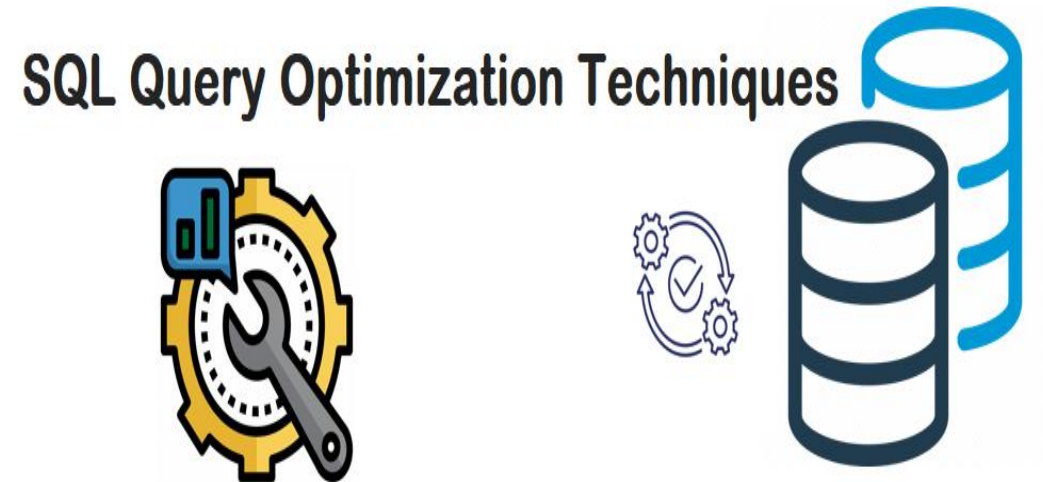
8 Write an SQL query to **display employees who do not receive any commission.**

9 Write an SQL query to **find the average salary of employees in department 20.**

10 Write an SQL query to **display employee names along with their manager numbers.**

Query Optimization

- To improve the performance of database queries by optimizing their execution plans.
- Techniques for Query Optimization
 - Index Usage
 - Join Optimization
 - Predicate Pushdown
 - Query Rewriting
 - Parallel Execution
 - Caching



Transaction

A **transaction** is a **sequence of database operations** (read, write, update, delete) that **must be executed completely or not at all**.

Example:

ATM Money Withdrawal

Steps:

1. Check balance
2. Deduct amount
3. Update balance
4. Dispense cash

→ All steps together form **one transaction**.

If any step fails → **entire transaction is cancelled (rollback)**.

Transaction Management

- a crucial aspect of database systems that ensures data consistency, integrity, and reliability.

ACID properties are fundamental aspects of Transaction Management

- Atomicity
- Consistency
- Isolation
- Durability

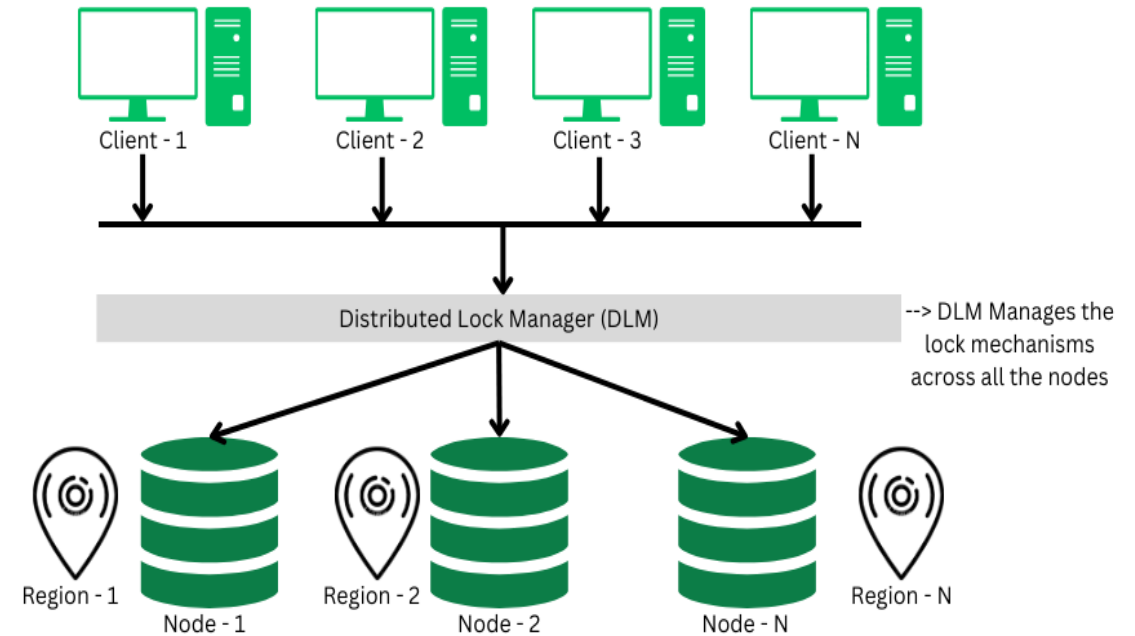
Property	Meaning	Example
Atomicity	All or nothing	Cash not given → no deduction
Consistency	Valid state to valid state	Balance never becomes negative
Isolation	Transactions independent	Two users withdraw safely
Durability	Data saved permanently	After commit, power failure doesn't lose data

Concurrency Control

Concurrency control is the process of managing **simultaneous execution of multiple transactions** in a database so that **data consistency, correctness, and isolation** are maintained.

In simple words:

It ensures that **many users can work on the database at the same time without corrupting data.**



Why is Concurrency Control Needed?

When multiple transactions access the same data at the same time, problems can occur like:

1.Lost Update

2.Dirty Read

3.Uncommitted Dependency

4.Inconsistent Data

Example:

Two users withdrawing money from the same bank account simultaneously can cause **wrong final balance** if concurrency control is not used.

Banking Example: ATM Withdrawals

Initial Balance = ₹10,000

Two customers withdraw money at the **same time**:

- **Transaction T1: Withdraw ₹4,000**
- **Transaction T2: Withdraw ₹3,000**

Case 1: Without Concurrency Control

(Problem Occurs)

Step-by-step:

T1 reads balance = 10000

T2 reads balance = 10000

T1 updates $\rightarrow 10000 - 4000 = 6000$

T2 updates $\rightarrow 10000 - 3000 = 7000$

✗ Final Balance = ₹7000

✓ Correct Balance should be = ₹3000

Case 2: With Concurrency Control (Using Locks)

Step-by-step using Locking Mechanism:

T1 gets EXCLUSIVE LOCK on balance

T1 reads 10000

T1 updates → 6000

T1 commits and releases lock

T2 now gets lock

T2 reads 6000

T2 updates → 3000

T2 commits

✓ **Final Balance = ₹3000 (Correct)**

Current Trends in DBMS

- Several current trends are shaping the development and adoption of DBMS technologies
 - Cloud-based Databases
 - Big Data Analytics
 - Real-time Data Processing
 - Microservices Architecture
 - Graph Databases
 - Blockchain Databases

Web Links For Reference (Complete Video Course)

- https://youtube.com/playlist?list=PLZ2ps__7DhBYc4jkUk_yQAjYEVFzVzhdU&si=2cdYZiJa6NVwbJ6s
- <https://youtube.com/playlist?list=PLIwC9bZ0rmjSkm1VRJROX4vP2YMIf4Ebh&si=kH6pymcwWOLcGtqP>
- https://youtube.com/playlist?list=PLBlnK6fEyqRi_CUQ-FXxgzKQ1dwr_ZJWZ&si=vRmw_BpdGuBIhDkb

Web Links For Reference (Study Material)

- <https://www.appsierra.com/blog/database-normalization-tutorial>
- <https://www.exasol.com/blog/what-is-a-relational-database/>
- <https://www.geeksforgeeks.org/dbms/>
- <https://db-book.com/slides-dir/>
- <https://www.w3schools.com/sql/>
- <https://www.javatpoint.com/dbms-tutorial>
- <https://byjus.com/govt-exams/database-management-system-dbms/>